COLLEGE / UNIVERSITY

Department of Information Technology / Computer Science

B.TECH — LECTURE NOTES

FUNDAMENTALS OF QUANTUM COMPUTING

UNIT II: Quantum Gates and Circuits

Course Unit: 300 minutes (5 hours)

Prepared in the same lecture-notes structure as the supplied sample document

Unit II — Syllabus Coverage

This unit develops the operational side of quantum computing: how quantum states are transformed by gates, how gates are composed into circuits, how universal gate sets are defined, and how standard circuits such as Bell-state preparation, teleportation, and superdense coding are constructed and analyzed.

Topics Covered

- Introduction: from quantum states to operations
- Reversible computing and the unitary requirement
- Pauli gates and single-qubit rotations
- Hadamard, phase, and T gates
- Multi-qubit controlled gates
- The quantum circuit model
- Universal gate sets and transpilation
- Worked circuit designs and simulation
- Unit recap and self-assessment

Unit Learning Outcomes

1. Explain why quantum gates are represented by unitary transformations.
2. Verify whether a small matrix is unitary.
3. Describe and apply the Pauli-X, Pauli-Y, and Pauli-Z gates.
4. Explain the Hadamard, S, T, and general phase gates.
5. Interpret quantum gates geometrically as transformations of qubit states.
6. Construct and analyze controlled operations such as CNOT, CZ, SWAP, Toffoli, and Fredkin.
7. Read and construct quantum-circuit diagrams and calculate circuit outputs.
8. Explain the meaning of circuit width, depth, and gate count.
9. Distinguish universal gate sets and explain why transpilation is needed on real hardware.
10. Build standard circuits for Bell states, teleportation, superdense coding, and simple state preparation.
11. Use a quantum simulator to verify circuit behavior and measurement results.

Part A: Introduction — From States to Operations

A.1 Quantum States versus Quantum Operations

Unit I introduced quantum states, amplitudes, measurement, and entanglement. Unit II focuses on how those states are deliberately transformed. A quantum gate is a physical or abstract operation that maps one valid quantum state to another valid quantum state.

For a closed quantum system, deterministic evolution is represented by a unitary matrix U. If $|\psi\rangle$ is the input state, the output is $|\psi'\rangle = U|\psi\rangle$.

A.1.1 The Gate–Circuit–Algorithm Connection

- States describe the information being processed.
- Gates transform quantum states.
- Circuits arrange gates in a sequence or parallel structure.
- Algorithms use carefully designed circuits to produce useful interference and measurement statistics.
- Hardware implements a native subset of gates, so compiled circuits may differ from their abstract algorithmic form.

A.2 Why Quantum Gates Must Be Unitary

A unitary matrix satisfies $U^\dagger U = I$. Consequently, if $|\psi\rangle$ is normalized, $U|\psi\rangle$ is also normalized. This preserves the total probability of measurement outcomes.

Unitary evolution is also reversible: $U^\dagger$ is the inverse of U. This reversibility distinguishes quantum gates from irreversible classical operations such as ordinary AND and OR.

A.2.1 Landauer's Principle and Reversible Computing

Classical logical operations can erase information. Landauer's principle connects irreversible information erasure with a minimum thermodynamic cost. Reversible computing avoids information loss at the logical level and provides useful conceptual background for quantum computing.

A.2.2 Reversible Classical Gates

- NOT is reversible.
- CNOT is reversible.
- Toffoli is reversible.
- Fredkin is reversible.

These gates help bridge classical reversible computation and quantum circuit design.

Worked Example A1: Checking Unitarity

Consider $X = [[0,1],[1,0]]$. Because $X^\dagger = X$ and $X^2 = I$, we have $X^\dagger X = I$. Therefore X is unitary.

For a candidate matrix U, the general procedure is: compute the conjugate transpose $U^\dagger$, multiply $U^\dagger U$, and check whether the result equals the identity matrix.

Part B: Single-Qubit Gates I — The Pauli Family

B.1 Identity Gate

The identity gate I leaves the state unchanged: $I|\psi\rangle = |\psi\rangle$. It is useful in circuit descriptions and in understanding operations that act on only selected qubits.

B.2 Pauli-X Gate

The Pauli-X gate is the quantum analogue of a NOT operation. Its matrix is $X = [[0,1],[1,0]]$. It exchanges the computational basis states: $X|0\rangle = |1\rangle$ and $X|1\rangle = |0\rangle$.

On the Bloch sphere, X corresponds to a rotation by π about the x-axis, up to the standard global-phase convention for representing the operation.

B.3 Pauli-Y Gate

The Pauli-Y matrix is $Y = [[0,-i],[i,0]]$. It maps $|0\rangle$ to $i|1\rangle$ and $|1\rangle$ to $-i|0\rangle$. The phase factors matter when the operation is part of a larger circuit.

Geometrically, Y corresponds to a π rotation about the y-axis, again understood up to global phase.

B.4 Pauli-Z Gate

The Pauli-Z matrix is $Z = [[1,0],[0,-1]]$. It leaves $|0\rangle$ unchanged and changes the sign of $|1\rangle$: $Z|0\rangle = |0\rangle$ and $Z|1\rangle = -|1\rangle$.

Z therefore changes relative phase without changing computational-basis probabilities when applied directly to $|0\rangle$ or $|1\rangle$.

B.5 Pauli Algebra

- $X^2 = Y^2 = Z^2 = I$.
- The Pauli matrices do not generally commute.
- Anticommutation relationships are important in quantum operator algebra.
- Products of Pauli operators can produce phase factors such as i or −i.

B.6 Rotation Gates

The Pauli matrices generate continuous single-qubit rotations. Common rotation gates are $R_x(\theta)$, $R_y(\theta)$, and $R_z(\theta)$. Their angles determine how far the state is rotated around the corresponding Bloch-sphere axis.

These rotations provide a convenient language for describing arbitrary single-qubit transformations and are important when mapping abstract circuits to hardware-native operations.

Worked Example B1: Applying X and Z

Starting from $|0\rangle$, applying X gives $|1\rangle$. Applying Z afterward gives $-|1\rangle$. The measurement probabilities in the computational basis remain deterministic: the outcome is still 1. The minus sign is a relative-phase effect and becomes relevant if later gates create interference.

Part C: Single-Qubit Gates II — Hadamard, Phase and T

C.1 Hadamard Gate

The Hadamard gate is H = (1/√2)[[1,1],[1,−1]]. It maps |0⟩ to |+⟩ and |1⟩ to |−⟩.

- H|0⟩ = (|0⟩ + |1⟩)/√2
- H|1⟩ = (|0⟩ − |1⟩)/√2

The Hadamard gate is also a basis transformation connecting the computational Z basis with the X basis. Since $H^2 = I$, applying H twice returns the original state.

C.2 Phase Gate S

The S gate applies a phase of π/2 to the |1⟩ component: S = diag(1,i). It leaves |0⟩ unchanged and maps |1⟩ to i|1⟩.

The identity $S^2 = Z$ connects phase operations with the Pauli-Z gate.

C.3 T Gate

The T gate applies a phase of π/4 to the |1⟩ component: T = diag(1,e^{iπ/4}). It is commonly called the π/8 gate because of its historical relation to rotation conventions.

The identity $T^2 = S$ shows the relationship between these phase gates.

C.4 General Phase Gate

A phase gate P(φ) can be written as diag(1,e^{iφ}). It changes relative phase without changing the computational-basis probability distribution of a state measured immediately afterward.

Worked Example C1: H–Z–H Identity

Using HZH = X, a circuit that applies H, then Z, then H has the same overall action as X. This is an example of a circuit identity and demonstrates that the same logical transformation can sometimes be expressed using different gate sequences.

C.5 Building Arbitrary Single-Qubit Unitaries

General single-qubit unitary transformations can be decomposed into rotations and phase factors. In practical quantum programming, a compiler or transpiler can replace a high-level operation with a sequence of gates supported by the target backend.

Part D: Multi-Qubit Gates

D.1 Controlled Operations

A controlled gate applies an operation to a target qubit depending on the state of a control qubit. The control-target structure is fundamental for creating correlations and entanglement.

D.1.1 CNOT

The controlled-NOT gate applies X to the target when the control is $|1\rangle$. Its computational-basis action is:

- $|00\rangle \rightarrow |00\rangle$
- $|01\rangle \rightarrow |01\rangle$
- $|10\rangle \rightarrow |11\rangle$
- $|11\rangle \rightarrow |10\rangle$

CNOT is both reversible and unitary. It is one of the most important two-qubit gates in quantum circuits.

D.2 Controlled-Z and Controlled-Phase

Controlled-Z applies a phase flip to the $|11\rangle$ component while leaving the other computational-basis states unchanged. Controlled-phase gates generalize this idea by applying a selected phase when the control and target conditions are satisfied.

D.3 SWAP Gate

The SWAP gate exchanges two qubits. It can be decomposed into three CNOT gates:

12. CNOT with qubit 1 controlling qubit 2.
13. CNOT with qubit 2 controlling qubit 1.
14. CNOT with qubit 1 controlling qubit 2 again.

Thus a higher-level SWAP operation can be expressed using a smaller universal gate family.

D.4 Toffoli and Fredkin Gates

The Toffoli or CCNOT gate flips a target qubit only when two control qubits are both 1. The Fredkin or CSWAP gate swaps two target qubits when a control qubit is 1.

These gates are useful in reversible computation and demonstrate how complex conditional logic can be constructed from quantum operations.

D.5 Controlled-U Construction

A general single-qubit unitary U can be promoted to a controlled-U operation. Such constructions provide a general mechanism for making one quantum operation conditional on another qubit.

Worked Example D1: H + CNOT Creates a Bell State

Start with $|00\rangle$. Apply H to the first qubit:

$|00\rangle \rightarrow (|00\rangle + |10\rangle)/\sqrt{2}$.

Apply CNOT with the first qubit as control:

$(|00\rangle + |10\rangle)/\sqrt{2} \rightarrow (|00\rangle + |11\rangle)/\sqrt{2} = |\Phi+\rangle$.

The CNOT converts the superposition into an entangled state. Measuring both qubits gives correlated 00 and 11 outcomes.

Part E: The Quantum Circuit Model

E.1 Reading Circuit Diagrams

- Horizontal wires represent qubits.
- Time normally flows from left to right.
- Gate symbols represent transformations applied to one or more qubits.
- Measurement symbols convert quantum information to classical results.
- Barriers can be used to organize circuits and sometimes constrain compiler optimization.

E.2 Series Composition

If U1 is followed by U2, the combined operation is $U2U1|\psi\rangle$. The rightmost operation acts first when the circuit is represented using conventional matrix multiplication.

This order is important when manually calculating circuit output.

E.3 Parallel Composition

When operations act independently on different qubits at the same time, tensor products represent the combined transformation. For example, applying H to each of two qubits is represented by $H \otimes H$.

E.4 Circuit Width, Depth, and Gate Count

- Width: number of qubits used by the circuit.
- Depth: number of sequential layers of operations, subject to the circuit's scheduling.
- Gate count: number of operations, often tracked by gate type.

Lower depth and fewer gates can be important on noisy hardware because errors can accumulate as circuits become longer and more complicated.

E.5 Computing a Two-Qubit Circuit by Matrix Multiplication

To calculate a small circuit manually, write the initial state vector, construct the matrix for each gate, take tensor products where required, and multiply the matrices in the correct order. For larger circuits, simulation software is normally used.

Worked Example E1: Two Hadamards

Starting from $|00\rangle$, applying H to each qubit gives $(H\otimes H)|00\rangle = (|00\rangle + |01\rangle + |10\rangle + |11\rangle)/2$. All four computational-basis outcomes therefore have equal probability under immediate computational-basis measurement.

E.6 Deferred and Implicit Measurement

Measurement can sometimes be conceptually moved later in a circuit when later operations do not depend on the classical measurement result. This is the idea behind deferred measurement. The principle is useful when reasoning about equivalent circuit structures, although actual hardware implementations may choose different measurement schedules.

Part F: Universal Gate Sets

F.1 Meaning of Universality

A set of quantum gates is universal if circuits built from that set can approximate arbitrary quantum operations to the desired accuracy. Universality does not mean every unitary is represented exactly by a finite circuit; continuous operations generally require approximation.

F.1.1 Common Universal Sets

- {H, T, CNOT} is a standard approximately universal gate set.
- {Rx, Ry, Rz, CNOT} provides another common universal construction.

Different hardware systems may expose different native gate sets, so a high-level circuit must often be transformed before execution.

F.2 Solovay–Kitaev Theorem — Intuition

The Solovay–Kitaev theorem provides a theoretical foundation for approximating arbitrary single-qubit unitaries efficiently using a finite universal gate set under suitable assumptions. The theorem is included here for conceptual understanding rather than detailed proof.

F.3 Clifford Group and Clifford+T

Clifford operations form an important family of quantum gates with useful algebraic and simulation properties. Adding T gates produces the Clifford+T family, which is universal. T gates are therefore important in fault-tolerant quantum computation and resource estimation.

F.4 Native Gates and Transpilation

Real quantum processors do not necessarily execute every gate in an abstract circuit directly. A transpiler rewrites the circuit into operations supported by the selected backend, while also respecting connectivity and hardware constraints.

- Map logical qubits to physical qubits.
- Rewrite gates into the backend's native operations.
- Insert routing operations when qubits are not directly connected.
- Optimize depth and gate count where possible.
- Preserve the intended computational behavior.

Part G: Worked Circuit Designs and Simulation

G.1 Bell-State Generator

The standard Bell-state circuit starts from $|00\rangle$, applies H to qubit 0, and then applies CNOT with qubit 0 as control and qubit 1 as target. The result is $|\Phi+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$.

G.1.1 All Four Bell States

- Apply I or X to the appropriate qubit before/after the basic Bell-state skeleton to generate the other Bell states.
- $|\Phi+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$
- $|\Phi-\rangle = (|00\rangle - |11\rangle)/\sqrt{2}$
- $|\Psi+\rangle = (|01\rangle + |10\rangle)/\sqrt{2}$
- $|\Psi-\rangle = (|01\rangle - |10\rangle)/\sqrt{2}$

G.2 Quantum Half-Adder

A quantum half-adder can use CNOT and Toffoli gates to encode the classical sum and carry relationships reversibly. CNOT can produce the XOR-like sum bit, while Toffoli can generate the AND-like carry into an additional target register.

G.2.1 Conceptual Mapping

- Sum: XOR relation can be implemented with CNOT operations.
- Carry: AND relation can be implemented reversibly with a Toffoli construction.
- Ancilla or target qubits may be required because reversible computation cannot simply erase intermediate information.

G.3 Quantum Teleportation

Quantum teleportation transfers an unknown single-qubit state using a shared entangled pair plus two classical bits of communication. The protocol does not transport the physical qubit itself and does not permit faster-than-light communication.

15. Prepare an unknown state $|\psi\rangle$.
16. Prepare an entangled Bell pair.
17. Entangle the unknown state with one member of the Bell pair.
18. Measure the first two qubits, producing two classical bits.
19. Use those classical bits to conditionally apply X and/or Z corrections to the remaining qubit.

After the correction, the remaining qubit is in the original state $|\psi\rangle$, while the original state has been destroyed by measurement.

G.4 Superdense Coding

Superdense coding uses a shared Bell pair to transmit two classical bits by sending one qubit. The sender applies one of four operations—typically I, X, Z, or XZ—to encode two classical bits, sends one qubit, and the receiver performs a Bell-basis decoding circuit.

- 00 → I
- 01 → X
- 10 → Z
- 11 → XZ (up to convention/order)

The protocol illustrates how shared entanglement changes the information capacity of a quantum communication procedure.

G.5 Preparing a Target State

State-preparation design asks: given a target state, which sequence of gates prepares it from a standard initial state such as $|0...0\rangle$? For a single qubit, rotations and phase gates can construct arbitrary states. For multi-qubit states, entangling gates are required when the target state is not separable.

G.6 AI-Assisted Circuit Optimization

AI-assisted tools can be used to suggest circuit simplifications, identify redundant operations, or compare alternative decompositions. Such suggestions should be checked against the original circuit using state-vector simulation, equivalence checking, or measurement statistics.

- Compare original and optimized circuit depth.
- Compare gate counts and two-qubit-gate counts.
- Verify that the output state or relevant measurement distribution is preserved.
- Check that hardware connectivity constraints are still satisfied.

Part H: Integrated Worked Examples

Worked Example 1: Apply a Gate Sequence

Start with $|0\rangle$ and apply H followed by Z followed by H. Since HZH = X, the final state is $X|0\rangle = |1\rangle$. A direct matrix calculation produces the same result.

Worked Example 2: CNOT on a Superposition

Start with $|+\rangle|0\rangle = (|00\rangle + |10\rangle)/\sqrt{2}$. Applying CNOT with the first qubit as control gives $(|00\rangle + |11\rangle)/\sqrt{2}$. The output is entangled, demonstrating that a two-qubit gate can convert a separable state into an entangled state.

Worked Example 3: SWAP Decomposition

A SWAP can be decomposed as CNOT(1→2), CNOT(2→1), CNOT(1→2). Applying the sequence to $|01\rangle$ produces $|10\rangle$. Applying it to $|10\rangle$ produces $|01\rangle$, confirming the intended exchange.

Worked Example 4: Circuit Depth versus Gate Count

Two circuits may contain the same number of gates but have different depths if some gates act on independent qubits and can be scheduled in parallel. On noisy hardware, reducing depth can be especially important because longer execution exposes the computation to more decoherence and gate errors.

Unit Summary

- Quantum gates transform quantum states and are represented by unitary matrices.
- Unitarity preserves normalization and makes closed-system quantum evolution reversible.
- Pauli-X, Y, and Z form a fundamental family of single-qubit operations.
- Hadamard creates equal superpositions from computational-basis states and connects the Z and X bases.
- S and T are phase gates; phase operations become observable through later interference.
- Rx, Ry, and Rz provide continuous single-qubit rotations.
- Controlled gates make operations conditional and enable entanglement generation.
- CNOT, CZ, SWAP, Toffoli, and Fredkin are important multi-qubit gates.
- Quantum circuits are compositions of gates arranged over qubit wires; matrix multiplication describes sequential composition and tensor products describe parallel action.
- Circuit width, depth, and gate count are important measures when designing executable quantum programs.
- Universal gate sets can approximate arbitrary quantum operations, while transpilers map abstract circuits to hardware-native operations and connectivity.
- Bell-state preparation, teleportation, superdense coding, and reversible arithmetic illustrate the circuit model in practice.

Review Questions

20. Why must a quantum gate be represented by a unitary matrix for closed-system evolution?
21. Define a unitary matrix and write the condition $U^\dagger U = I$.
22. What is the relationship between reversibility and quantum computation?
23. State the matrices for X, Y, and Z.
24. Describe the action of X on $|0\rangle$ and $|1\rangle$.
25. How does Z affect the relative phase of a qubit?
26. Why are Pauli operators important in quantum computing?
27. What are Rx, Ry, and Rz rotations?
28. Write the Hadamard matrix and describe its action on $|0\rangle$ and $|1\rangle$.
29. What are the S and T gates and how are they related to Z and S?
30. Explain the difference between a global phase and a phase that can affect interference.
31. What is a controlled operation?
32. Give the truth-table action of CNOT.
33. How can a SWAP gate be constructed from CNOT gates?
34. What is a Toffoli gate and why is it useful for reversible computation?
35. What is meant by circuit width, circuit depth, and gate count?
36. How are sequential gates represented mathematically?
37. How are parallel gates on independent qubits represented?
38. What does it mean for a gate set to be universal?
39. What is the role of the Solovay–Kitaev theorem?
40. Why is transpilation necessary on real quantum hardware?
41. How does H followed by CNOT generate a Bell state?
42. Describe the main steps of quantum teleportation.
43. How does superdense coding use entanglement?
44. Why can a circuit with fewer sequential layers be advantageous on noisy hardware?
45. How would you verify that an optimized circuit is equivalent to the original circuit?

Self-Assessment Checklist

- I can verify whether a 2×2 matrix is unitary.
- I can calculate the result of applying X, Y, Z, or H to a simple state.
- I can explain the effect of a phase gate.
- I can construct a Bell state using H and CNOT.
- I can read a controlled-gate circuit.
- I can calculate the output of a small circuit using matrix multiplication.
- I can distinguish circuit width, depth, and gate count.
- I can explain what makes a gate set universal.
- I can explain why transpilation is required for real hardware.
- I can describe the logic of teleportation and superdense coding.
- I can use a simulator to compare state vectors and measurement distributions.

Suggested Mini-Lab

Create a notebook with four experiments: (1) verify X, H, and Z actions on single-qubit states; (2) build all four Bell states and record their measurement distributions; (3) compare a high-level circuit with a transpiled/decomposed version and record depth and gate counts; (4) implement a simple teleportation circuit and verify the output state for several input states. Document the theoretical prediction, simulator result, and explanation of any differences.